Package 'ASRgwas'

November 10, 2022

Type Package

Title ASRgwas an R package to perform Genome-Wide Association Studies (GWAS)

Version 1.0.0

Description ASRgwas is a comprehensive R package designed to perform Genome-Wide Association Studies (GWAS) using the modelling flexibility available in ASReml-R. This library assists with preparing and auditing phenotypic and genomic data; fitting GWAS models and identifying significant markers; and evaluating and exploring output.

License MIT + file LICENSE

Depends R (>= 4.0.0)

Imports cli,
data.table,
ggplot2,
ggrepel,
Matrix,
methods,
parallel,
Rcpp

Suggests knitr,
rmarkdown

Enhances asreml,
ASRgenomics

LinkingTo Rcpp,
RcppArmadillo

VignetteBuilder knitr

Encoding UTF-8

LazyData true

RoxygenNote 7.2.1

R topics documented:

`ASRgwas` *ASRgwas: An R package to perform complex Genome-Wide Association Studies (GWAS)*

Description

`ASRgwas` is a complete package designed to perform Genome-Wide Association Studies (GWAS) using the modelling flexibility available in `ASReml-R`. The aim of this package is to perform in an efficient and reliable way these types of analyses. This library assists preparing data and matrices, and verifies that they are adequate to perform GWAS. In addition, it has a set of complementary functions to be used for *post-GWAS* analyses to help on interpretation, use of the output information, and obtaining graphical outputs.

The main tasks considered within `ASRgwas` are:

- Preparing and auditing phenotypic and genomic data.
- Fitting GWAS models and identifying significant markers.
- Evaluating results and generating tables and graphical output.

The main function `gwas.asreml` uses `ASReml-R` to fit a *base model* which will be used as the starting point for the GWAS procedure. At this point, two paths can be chosen for carrying out GWAS to determine marker significance. In the first, variance components and design matrices are extracted from the fitted model and used to implement the approximated, but fast, algorithm called Population Parameter Previously Determined (P3D; implemented for Gaussian data). In the second path, the *base model* is updated for each marker individually using **asreml**'s update.asreml function; this is an exact procedure but slower and works well with phenotypic responses that follow a Normal or binomial distribution.

The package has unique features including the ability to work with complex mixed models, (any number of covariates, fixed and/or random structures), heterogeneous error variances, replicated data, missing values on the genomic matrix, and binomial distribution of phenotypic responses.

Details

A brief summary of the functions implemented in `ASRgwas` are described below (presented in order of proposed workflow):

- `pre.gwas`: Prepares and audits phenotypic and genomic data for GWAS.
- `audit.gwas`: Checks user' provided datasets and matrices prior to GWAS.
- `gwas.asreml`: Performs fitting of GWAS model using **asreml**.

- `select.marker`: Selects final set of GWAS markers using backward elimination.
- `qq.plot`: Draws quantile-quantile plots for GWAS marker data results.
- `manhattan.plot`: Draws Manhattan plots for GWAS marker data results.
- `map.plot`: Generates a graphical representation of the genetic map.
- `marker.plot`: Generates phenotype-by-genotype plots for selected markers.

Author(s)

VSN International Ltd.

`audit.gwas` *Checks/audits datasets and matrices prior to GWAS*

Description

Implements checks and basic exploratory data analysis (EDA) on phenotypic and genomic datasets in order to ensure it will work adequately on downstream GWAS model fits using function `gwas.asreml`. In addition, reports mismatches within and between datasets in order to help detect and correct inconsistencies.

This function can be used in place of `pre.gwas` in order to provide `gwas.asreml` with all the required elements. These can be constructed in other routines, libraries or your own code. This function will ensure that all attributes and elements are suitable for use with `gwas.asreml`.

Usage

```
audit.gwas(
  pheno.data = NULL,
  indiv = NULL,
  resp = NULL,
  Kinv = NULL,
  geno.data = NULL,
  map.data = NULL,
  Q = NULL,
  message = TRUE
)
```

Arguments

`pheno.data`	A data frame with all relevant columns (factors and covariates) and phenotypic responses to be used (default = `NULL`).
`indiv`	A character with the name of the column in `pheno.data` with the identification of treatments (genotypes) (default = `NULL`).
`resp`	A character with the name of the numeric variable in `pheno.data` with the response variable (default = `NULL`).
`Kinv`	A list of matrices representing the **inverse** of the genomic relationship matrix $\boldsymbol{Kinv}$. The matrices must be in sparse form (default = `NULL`).
`geno.data`	A matrix with SNP data of form $n \times p$, with n individuals and p markers. Individual and marker names are assigned to `rownames` and `colnames`, respectively. SNP data is coded as: 0, 1, 2 (default = `NULL`).

map.data	A data frame with each marker's name, chromosome, and position. Variable names **must** be: "marker", "chrom", and "pos" (default = NULL).
Q	A matrix of size n x nv, with nv population structure-related covariates (e.g., principal component scores; default = NULL).
message	If TRUE diagnostic messages are printed on screen (default = TRUE).

Value

A list containing information about the dataset, and summary statistics for the phenotypic, relationship and molecular data.

Examples

```
# Prepare Apricot datasets (simple run).
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot)

# Audit data frames and matrices prior to GWAS.
audit <- audit.gwas(
 pheno.data = gwas.data$pheno.data, resp = "Sucrose", indiv = "Ind",
 Kinv = gwas.data$Kinv, geno.data = gwas.data$geno.data, Q = gwas.data$Q)
audit$study.stats
audit$gen.stats
audit$n.genotypes
audit$n.markers
audit$ind.callrate.stats
audit$marker.callrate.stats
audit$maf.stats
audit$Kinv.condition
```

---

geno.apricot *Genotypic data for apricot dataset published by Nsibi* et al. *(2020)*

Description

Genotypic data for a total of 60,991 SNPs (coded as 0, 1, 2 and NA for missing) on 155 genotypes of apricot (*Prunus armeniaca*). Dataset obtained from supplementary material in Nsibi *et al.* (2020).

For more information refer to the original publication.

Usage

```
geno.apricot
```

Format

matrix

References

Nsibi M., Gouble B., Bureau S., Flutre T., Sauvage C., Audergon J.M., Regnard J.L. 2020. Adoption and optimization of genomic selection to sustain breeding for Apricot fruit quality. *G3 Genes, Genomes, Genet.* 10(12):4513-4529.

Examples

```
geno.apricot[1:5, 1:5]
```

`gwas.asreml` *Fits Genome-wide Association Studies (GWAS) model*

Description

Main function of **ASRgwas** that extends the capabilities of **asreml** to perform Genome-wide Association Studies.

Usage

```
gwas.asreml(
  pheno.data = NULL,
  resp = NULL,
  gen = NULL,
  Kinv = NULL,
  Q = NULL,
  npc = 10L,
  cov = NULL,
  fixedf = NULL,
  randomf = NULL,
  residual = NULL,
  weights = NULL,
  family = c("gaussian", "binomial"),
  dispersion = 1,
  total = NULL,
  workspace = "1Gb",
  mod = NULL,
  geno.data = NULL,
  map.data = NULL,
  pvalue.thr = 0.05,
  bonferroni = TRUE,
  P3D = TRUE,
  maxiter.update = 2L,
  inverse.update = -1L,
  threads = detectCores() - 1,
  obj.parallel = NULL,
  message = TRUE
)
```

Arguments

`pheno.data`	A data frame with all relevant columns (factors and covariates) and phenotypic responses to be used (default = `NULL`).
`resp`	A character with the name of the numeric variable in `pheno.data` with the response variable (default = `NULL`).
`gen`	A character (vector) with the names of the columns in `pheno.data` with the genotypes for the "genetic" part of the model (default = `NULL`).

Kinv — A list of matrices representing the **inverse** of the genomic relationship matrix $\boldsymbol{Kinv}$. The matrices must be in sparse form (default = NULL).

Q — A matrix of size n x nv, with nv population structure-related covariates (e.g., principal component scores). The number of components to be included in the fitted model is determined by npc (default = NULL).

npc — The number of components (columns) to be used from the $\boldsymbol{Q}$ matrix (default = 10L).

cov — A character vector with the names of the numeric variables in pheno.data to be considered as covariates in the GWAS model (default = NULL).

fixedf — A character vector with the names of the factors in pheno.data to be considered as fixed effects in the GWAS model. Interactions can be added as "factor1:factor2" (default = NULL).

randomf — A character vector with the names of the factors in pheno.data to be considered as random effects in the GWAS model. Interactions can be added as "factor1:factor2". Interactions with gen factor can also be added here (e.g., "gen:factor1") (random by interaction with gen). Unique model terms in randomf should not appear in gen (default = NULL).

residual — A character with the name of a variable in pheno.data to be used as a conditional factor for specifying heterogeneous residuals according to the levels of this factor. The pheno.data will be internally ordered by this factor (default = NULL).

weights — A character with the name of the numeric variable in pheno.data with the weights of each prediction (i.e. mean estimate) (default = NULL).

family — A character indicating the family of the distribution for resp. Options are: "gaussian" and "binomial" (default = "gaussian").

dispersion — A numeric value with the dispersion parameter (used if family = "binomial") (default = 1).

total — A character with the name of a numeric variable in pheno.data with the total counts (if family = "binomial"). If NULL, count is taken as 1 (default = NULL).

workspace — Specifies the workspace to be used by the asreml function. Note that large workspaces will result in much longer processing times if P3D is false (default = "1Gb").

mod — A pre-fitted **asreml** model object. Providing a pre-fitted model works for Gaussian data if P3D = FALSE and binomial data (default = NULL).

geno.data — A matrix with SNP data of form $n \times p$, with n individuals and p markers. Individual and marker names are assigned to rownames and colnames, respectively. SNP data is coded as: 0, 1, 2 (default = NULL).

map.data — A data frame with each marker's name, chromosome, and position. Variable names **must** be: "marker", "chrom", and "pos" (default = NULL).

pvalue.thr — A numeric value with the $p - value$ threshold to identify significant markers (default = 0.05)

bonferroni — If TRUE the Bonferroni correction is used with expression $1-(1-pvalue.thr)^{1/p} \sim pvalue.thr/p$, where $pvalue.thr$ is based on the argument (pvalue.thr) and p is the number of markers (default = TRUE).

P3D — If TRUE the "Population Parameters Previously Determined" algorithm will be used. If FALSE the variance components are allowed to vary while fitting each marker individually. Setting P3D = TRUE results in considerable speed gain for family = "gaussian", but not for family = "binomial" (default = TRUE).

`maxiter.update` A numeric value indicating the number of iterations to be used in `update.asreml` method if requested (default = 2).

`inverse.update` An integer indicating the number of Schulz-type iterative updates to be applied to the inverse of $\boldsymbol{V}$ (phenotypic variance matrix) if `geno.data` has missing values when running a Gaussian model under P3D. The larger the value the better the approximation (under certain conditions). If `inverse.update = -1` then the Woodbury's method is used instead. If `inverse.update = 0` is used, then Schulz's procedure is performed with no iterations (no update), which is less precise but faster to run. See details for more information (default = `-1`).

`threads` An integer with the number of thread to be used in parallel processing (default = `parallel::detectCores() - 1`; i.e., total number of threads available minus one).

`obj.parallel` A character vector indicating which objects to be passed to the function that must be exported to clusters (e.g. objects passed to `Kinv` argument). If nothing is passed, the algorithm will try to identify the required objects (default = `NULL`).

`message` If `TRUE` diagnostic messages are printed on screen (default = `TRUE`).

Details

Some relevant considerations are presented below.

Marker only models (no $\boldsymbol{Q}$ or $\boldsymbol{K}$) can be fitted by specifying `gen`, `Q`, and `Kinv` as `NULL`.

The factors passed to `gen` are considered random, as are the ones passed via `randomf`. The arguments are separated so it is clear which ones should be on the numerator (`gen`) and denominator (`gen + randomf`) of the heritability (or repeatability) formula. Interactions with `gen` factors should be included in `randomf` and will be always added to the denominator. No interactions between `gen` factors are allowed.

If there is more than one type of genetic effect to be modeled (but associated with different genomic matrices), the column with genotype identifications in `pheno.data` should be copied with a different name so there is one for each genetic effect considered.

If relationship matrices are associated to `gen` factors, they should be passed using `Kinv`. The name of objects in `Kinv` **must** match the values passed to `gen`. For example, to model additive and dominance effects, the following code can be used: `gen = c("genA", "genD")`, where `"genA"` and `"genD"` are columns in `pheno.data` with genotype identifiers; and `Kinv = list("genA" = KinvA, "genD" = KinvD)`, where `KinvA` and `KinvD` are inverse matrices of additive and dominance relationships in sparse form. If a factor is passed to `gen` but not to `Kinv`, it is regarded as having independent levels (i.e. `id/idv` variance structure). Also, all factors names passed to the command `Kinv` **must** be present in the command `gen`.

Row and column names are checked for matching between and within lists, and between datasets. In the case of the sparse form `Kinv`, attribute names `rowNames` and `colNames` are used to perform this verification.

Any number of fixed (factors and covariates) and random effects can be passed, but only single trait analysis has been implemented so far.

Missing values in `resp` and `weights` are allowed. They are dealt with by eliminating the observations (rows) prior to the function call.

Missing values on the marker are allowed in both `P3D = TRUE` and `P3D = FALSE`. If `P3D = FALSE`, **asreml** deals with the missing data. Two algorithms (Schulz and Woodbury) have been implemented to deal with missing data by adjusting the inverse of the $\boldsymbol{V}$ (phenotypic variance) matrix if `P3D = TRUE`. It is likely that Woodbury's method will be faster and more precise than Schulz's method with

one or more iterations. In addition, Schulz's method is highly dependent on matrix stability, while Woodbury's method is more robust. Note that the default is Woodbury's method.

The variance explained by each marker present in `gwas.all/gwas.sel` is calculated via $expl.var = 100 \times [2 \times maf \times (1-maf) \times effect^2]/var(resp)$. The MAF (`maf`; and consequently the variance explained `expl.var`) are only correct for `geno.data` with additive genotype coding.

The calculation of the $\boldsymbol{V}$ (phenotypic variance) matrix required by Gaussian P3D is based on the code previously implemented for the library **asremlPlus** (Brien, 2021).

Value

A list of class `gwas.asreml` with several outputs from a GWAS model fit:

- `call`: An object of class `call` with the code used to fit `mod` in **asreml**.
- `gwas.all`: A data frame with map (`marker`, `chrom`, `pos`), minor allele frequency (`maf`), `effect`, `z.ratio`, `p.value`, and explained variance (`expl.var`; see details) of all tested markers.
- `gwas.sel`: a subset of `gwas.all` with significant markers based on `pvalue.thr` and/or Bonferroni.
- `mod`: The **asreml** object base model fitted with the provided data.
- `aov`: The ANOVA table of the fitted base model.
- `heritability`: The estimated heritability value for `h2.vc` and `h2.pev` definitions.

References

Brien C. 2021. asremlPlus: Augments 'ASReml-R' in fitting mixed models and packages generally in exploring prediction differences. R package version 4.3-31, <https://CRAN.R-project.org/package=asremlPlus>.

Hager, W.W. 1989. Updating the inverse of a matrix. *SIAM Review* 31(2):221–239.

Petković M.D. 2014. Generalized Schultz iterative methods for the computation of outer inverses. *Computers & Mathematics with Applications* 67(10).

Zhang, Z. et al. 2010. Mixed linear model approach adapted for genome-wide association studies. *Nature Genetics*, 42(4):355–360.

Examples

```
# Prepare Apricot dataset.
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot, map.data = map.apricot,
  maf = 0.05, heterozygosity = 0.9, Fis = 0.5,
  marker.callrate = 0.40, impute = TRUE)

# Fit model to see how good it is.
gwasA <- gwas.asreml(
  pheno.data = gwas.data$pheno.data,
  resp = "Sucrose",
  gen = "Ind",
  fixedf = "Lots",
  residual = "Lots",
  Kinv = gwas.data$Kinv,
  Q = gwas.data$Q,
  npc = 3)
```

```
# Check the call.
gwasA$call

# Check heritability/repeatability.
gwasA$heritability

# Check the ANOVA table.
gwasA$aov

# Fit the GWAS for that model.
gwasA <- gwas.asreml(
  pheno.data = gwas.data$pheno.data,
  resp = "Sucrose",
  gen = "Ind",
  fixedf = "Lots",
  residual = "Lots",
  Kinv = gwas.data$Kinv,
  Q = gwas.data$Q,
  npc = 3,
  geno.data = gwas.data$geno.data,
  map.data = gwas.data$map.data,
  pvalue.thr = 0.0005,
  bonferroni = FALSE,
  workspace = "2Gb")

# The quantile-quantile plot.
qq.plot(gwas.table = gwasA$gwas.all)

# The Manhattan plot.
manhattan.plot(gwas.table = gwasA$gwas.all, padding = 2e6)

# Statistics for all markers.
head(gwasA$gwas.all)

# Statistics for only significant markers.
head(gwasA$gwas.sel)
```

`manhattan.plot` *Draws one or more Manhattan plots for GWAS marker data results*

Description

Generates one or more Manhattan plots for visualization.

Usage

```
manhattan.plot(
  gwas.table = NULL,
  pvalue.thr = NULL,
  tag.table = NULL,
  tag.repel = TRUE,
  point.colour = "chrom",
  point.size = 1,
  point.alpha = 1,
```

```
  collate = TRUE,
  padding = 0,
  gwas.index = NULL,
  facet.main = NULL,
  facet.secondary = NULL,
  facet.dim = NULL,
  scales = "free_y",
  legend.position = "none",
  message = TRUE
)
```

Arguments

gwas.table	A data frame of one or more (vertically stacked) GWAS analysis with (as a minimum) the following variables: "marker", "chrom", "pos", and "p.value" of markers (default = NULL).
pvalue.thr	A numeric value with the p-value threshold to identify significant markers (e.g. 5e-4). The value will be transformed using -log10(pvalue.thr) for plotting (default = NULL).
tag.table	A data frame of tags (annotations) for scores. The columns "marker" and "tag" are required. The column "tag" should contain the text to be annotated in each marker score (e.g., "Candidate region 3"). Columns "nudge.x" and "nudge.y" are optional to inform position nudges on the text and should be on the base pair number and log(p-value) scale, respectively (default = NULL).
tag.repel	If TRUE, nudge.x and nudge.y from tag.table are ignored and tags are automatically repelled (default = TRUE).
point.colour	A character with a colour name (e.g. "blue") or a column name from gwas.table to be used as the index for point colouring (default = "chrom").
point.size	A numeric value (greater than 0) indicating the point size (default = 1).
point.alpha	A numeric value indicating the points' transparency (default = 1).
collate	If TRUE the distance between the last marker of a chromosome and the first marker of the next chromosome is set to 1 base pair (default = TRUE).
padding	A numeric value that will be used as the distance between chromosomes (default = 0).
gwas.index	A character indicating a column name in gwas.table identifying each GWAS analysis. This is only necessary when more than one GWAS analysis is present in gwas.table (default = NULL).
facet.main	A character indicating a column name in gwas.table to be used for the main faceting of panels (if more than one GWAS analysis is present) (default = NULL).
facet.secondary	A character indicating a column name in gwas.table to be used as secondary faceting of panels (if more than one GWAS analysis is present) (default = NULL).
facet.dim	A numeric vector indicating the number of rows and columns to be used in the faceting process, e.g. c(4, 1), for 4 rows and 1 column. This argument only works if facet.main is provided, and works best if facet.secondary is NULL (default = NULL).
scales	A character indicating if the scales/levels in x and y axis should be the same values across facets (panels). Options are: "fixed" "free_x", "free_y", and "free". See facet_wrap for more information (default = "free_y").

legend.position
: A character indicating the position for the legend in the displayed plot. Options are: "bottom", "top", "left", "right", "none", or the corresponding vector of coordinates (such as: c(0.95, 0.05) for x and y, respectively) (default = "none").

message
: If TRUE diagnostic messages are printed on screen (default = TRUE).

Value

A ggplot object containing one or more Manhattan plots.

Examples

```
# Prepare Apricot dataset.
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot, map.data = map.apricot, maf = 0.05,
  marker.callrate = 0.40, impute = TRUE)

# Run GWAS.
gwasA <- gwas.asreml(
  pheno.data = gwas.data$pheno.data, resp = "Sucrose", gen = "Ind",
  fixedf = "Lots", residual = "Lots",
  Kinv = gwas.data$Kinv, Q = gwas.data$Q, npc = 3,
  geno.data = gwas.data$geno.data, map.data = gwas.data$map.data,
  pvalue.thr = 0.0005, bonferroni = FALSE, workspace = "2Gb")

# Simple Manhattan plot call.
manhattan.plot(gwas.table = gwasA$gwas.all)

# Manipulate some features.
manhattan.plot(
  gwas.table = gwasA$gwas.all,
  pvalue.thr = 0.0005, point.alpha = 0.80, padding = 3e6)

# Adding tags.
gwasA$gwas.sel$tag <-
  c("Putative gene 1", "Putative gene 2",
    "Putative gene 3", "Putative gene 4", "Putative gene 5")

manhattan.plot(
  gwas.table = gwasA$gwas.all,
  pvalue.thr = 0.0005, point.alpha = 0.80, padding = 3e6,
  tag.table = gwasA$gwas.sel, tag.repel = TRUE)
```

---

map.apricot *Genomic map for apricot dataset published by Nsibi* et al. *(2020)*

Description

Genomic map for a total of 60,991 SNPs with marker names (marker), chromosomes (chrom), and positions (pos). Dataset obtained from supplementary material in Nsibi *et al.* (2020).

For more information refer to the original publication.

Usage

```
map.apricot
```

Format

data frame

References

Nsibi M., Gouble B., Bureau S., Flutre T., Sauvage C., Audergon J.M., Regnard J.L. 2020. Adoption and optimization of genomic selection to sustain breeding for Apricot fruit quality. *G3 Genes, Genomes, Genet.* 10(12):4513-4529.

Examples

```
map.apricot[1:5,]
```

---

map.plot *Draws the genetic map*

Description

Generates the genetic map for visualization.

Usage

```
map.plot(
  map.data = NULL,
  tag.markers = NULL,
  tag.repel = TRUE,
  tag.colour = "black",
  tag.size = 1.8,
  marker.colour = "#105E26",
  marker.width = 0.05,
  marker.alpha = 0.08,
  chrom.colour = "#DEDEDE",
  chrom.width = 4,
  chrom.contour = "#DEDEDE",
  message = TRUE
)
```

Arguments

map.data	A data frame containing the map information ("marker", "chrom", and "pos") (default = NULL).
tag.markers	A (non-mandatory) vector with the identification (name) of markers to be highlighted in the map (default = NULL).
tag.repel	If TRUE tags are automatically repelled (default = TRUE).
tag.colour	A character with a colour name (e.g. "black") to be used in the colouring of selected markers labels (default = "black").

tag.size	A numeric value (>= 0) indicating label sizes for selected marker names (default = 1.3).
marker.colour	A character with a colour name (e.g. "blue") to be used in marker colouring (default = "#105E26").
marker.width	A numeric value (>= 0) indicating the marker width (represented by a line; default = 0.05).
marker.alpha	A numeric value indicating the markers' transparency (default = 0.08).
chrom.colour	A character with a colour name (e.g. "white") to be used in inner chromosome colouring (default = "#DEDEDE").
chrom.width	A numeric value (> 0) indicating the chromosome width (default = 4).
chrom.contour	A character with a colour name (e.g. "#DEDEDE") to be used in outer chromosome colouring (default = "black").
message	If TRUE diagnostic messages are printed on screen (default = TRUE).

Details

This function was developed in collaboration with Khaled Al-Shamaa (ICARDA/CGIAR).

Value

A ggplot object containing the genetic map.

Examples

```
# Prepare Apricot dataset.
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot, map.data = map.apricot, maf = 0.05,
  marker.callrate = 0.40, impute = TRUE)

# Run GWAS.
gwasA <- gwas.asreml(
  pheno.data = gwas.data$pheno.data, resp = "Sucrose", gen = "Ind",
  fixedf = "Lots", residual = "Lots",
  Kinv = gwas.data$Kinv, Q = gwas.data$Q, npc = 3,
  geno.data = gwas.data$geno.data, map.data = gwas.data$map.data,
  pvalue.thr = 0.0005, bonferroni = FALSE, workspace = "2Gb")

# Get genetic map with the significant markers.
map.plot(map.data = gwasA$gwas.all,
  tag.markers = gwasA$gwas.sel$marker)

# Customizing a bit.
map.plot(map.data = gwasA$gwas.all,
  tag.markers = gwasA$gwas.sel$marker,
  marker.colour = "white", marker.width = 0.2,
  marker.alpha = 0.1, chrom.colour = "grey",
  chrom.contour = "grey", tag.colour = "#0072B2")
```

`marker.plot` *Generates phenotype-by-genotype plots for selected markers*

Description

Generates a plot, based on a given set of markers, for visualizing the phenotype in relation to the underlying genotypes. Both Gaussian and binomial data are supported.

Usage

```
marker.plot(
  pheno.data = NULL,
  indiv = NULL,
  resp = NULL,
  geno.data.sel = NULL,
  type.plot = c("boxplot", "violin"),
  family = c("gaussian", "binomial"),
  sample.label = FALSE,
  sample.width = FALSE,
  maf.colour = FALSE,
  facet.dim = NULL,
  scales = NULL,
  message = TRUE
)
```

Arguments

`pheno.data`	A data frame with all relevant columns (factors and covariates) and phenotypic responses to be used (default = `NULL`).
`indiv`	A character with the name of the column in `pheno.data` with the identification of treatments (genotypes) (default = `NULL`).
`resp`	A character with the name of the numeric variable in `pheno.data` with the response variable (default = `NULL`).
`geno.data.sel`	A matrix with SNP data of form $n \times s$, with n individuals and s selected markers. Individual and marker names are assigned to `rownames` and `colnames`, respectively. SNP data is coded as: 0, 1, 2 (default = `NULL`).
`type.plot`	A character indicating the type of plot. Options are: `"boxplot"` and `"violin"` plot (default = `"boxplot"`).
`family`	A character indicating the family of the distribution for `resp`. Options are: `"gaussian"` and `"binomial"` (default = `"gaussian"`).
`sample.label`	If `TRUE` the sample size of each genotypic state is added to the x axis (default = `FALSE`).
`sample.width`	If `TRUE` the size of the boxplots will be proportional to sample size (default = `FALSE`).
`maf.colour`	If `TRUE` the plot colour will be proportional to the minor allele frequency (default = `FALSE`).
`facet.dim`	A numeric vector indicating the number of rows and columns to be used in the faceting process, e.g. `c(4, 1)` for 4 rows and 1 column (default = `NULL`).

scales	A character indicating if the scales/levels in x and y axis should be the same values across facets (panels). Options are: "fixed" "free_x", "free_y" and "free". See facet_wrap for more information (default = NULL).
message	If TRUE diagnostic messages are printed on screen (default = TRUE).

Details

These plots are useful to verify and explore the validity of reported associations. For example: 1) a linear change is expected on significant markers for additive action, 2) a non-linear pattern might indicate non-additive action, 3) genotype classes might differ on their phenotypic distribution (e.g. heterogeneity of variances), and 4) the influence of low MAF is clear and might indicate significant associations by chance.

Check provided references for examples of these plots and their interpretation.

Value

A ggplot object containing one or more phenotype-by-genotype plots.

References

Croteau-Chonka D.C., Rogers A.J., Raj T., McGeachie M.J., Qiu W., et al. 2015. Expression quantitative trait loci information improves predictive modeling of disease relevance of non-coding genetic variation. *PLOS ONE* 10(10):e0140758.

Galli G., Alves F.C., Morosini J.S., and Fritsche-Neto R. 2020. On the usefulness of parental lines GWAS for predicting low heritability traits in tropical maize hybrids. *PLOS ONE* 15(2):e0228724.

Examples

```
# Prepare Apricot dataset.
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot, map.data = map.apricot, maf = 0.05,
  marker.callrate = 0.40, impute = TRUE)

# Run GWAS.
gwasA <- gwas.asreml(
  pheno.data = gwas.data$pheno.data, resp = "Sucrose", gen = "Ind",
  fixedf = "Lots", residual = "Lots",
  Kinv = gwas.data$Kinv, Q = gwas.data$Q, npc = 3,
  geno.data = gwas.data$geno.data, map.data = gwas.data$map.data,
  pvalue.thr = 0.0005, bonferroni = FALSE, workspace = "2Gb")

# Get name and data of significant markers.
sign.markers <- gwasA$gwas.sel$marker
geno.data.sel <- gwas.data$geno.data[, sign.markers]

# Simple marker plot call.
marker.plot(
 pheno.data = gwas.data$pheno.data, indiv = "Ind", resp = c("Sucrose"),
 geno.data.sel = geno.data.sel)

# Manipulate some features (including a trait).
marker.plot(
 pheno.data = gwas.data$pheno.data, indiv = "Ind",
 resp = c("Sucrose", "Ethylene"), geno.data.sel = geno.data.sel,
```

```
sample.label = TRUE, sample.width = TRUE, maf.colour = TRUE,
scales = "free")
```

`pheno.apricot` *Raw phenotypic information for apricot dataset published by Nsibi* et al. *(2020)*

Description

The plant material consists of a F1 generation from a cross between cultivars "Goldrich" and "Moniqui". The progeny is composed of 153 individuals evaluated at the INRAE experimental field in southern France. Only data for year 2006 is included here.

Fruits were collected and divided into three homogenous (maturity) lots of four fruits per genotype. Fruits were then measured for weight (`F.Weight`), ground colour (`Hue.g`), ethylene (`Ethylene`), refractive index (solid soluble content; `RI`), titratable acidity (`TA`), glucose (`Glucose`), fructose (`Fructose`), citric acid (`Citric.A.`), and malic acid (`Malic.A.`).

For more information refer to the original publication.

Usage

```
pheno.apricot
```

Format

data frame

References

Nsibi M., Gouble B., Bureau S., Flutre T., Sauvage C., Audergon J.M., Regnard J.L. 2020. Adoption and optimization of genomic selection to sustain breeding for Apricot fruit quality. *G3 Genes, Genomes, Genet.* 10(12):4513-4529.

Examples

```
head(pheno.apricot)
```

`pre.gwas` *Prepares and audits phenotypic and genomic data for GWAS*

Description

A function to check the phenotypic and genomic data and to prepare the required data frames for downstream GWAS with `gwas.asreml`. The mandatory input is `pheno.data`, `indiv`, and `geno.data`. The map (if not provided), and the matrices $\boldsymbol{K}$, $\boldsymbol{Kinv}$ and $\boldsymbol{Q}$ will be generated.

Usage

```
pre.gwas(
  pheno.data = NULL,
  indiv = NULL,
  geno.data = NULL,
  resp = NULL,
  weights = NULL,
  map.data = NULL,
  rename.markers = TRUE,
  Q.method = c("K", "geno.data"),
  K = NULL,
  return.K = FALSE,
  message = TRUE,
  ...
)
```

Arguments

`pheno.data`	A data frame with all relevant columns (factors and covariates) and one or more phenotypic responses to be used (default = `NULL`).
`indiv`	A character with the name of the column in `pheno.data` with the identification of treatments (genotypes) (default = `NULL`).
`geno.data`	A matrix with SNP data of form $n \times p$, with n individuals and p markers. Individual and marker names are assigned to `rownames` and `colnames`, respectively. SNP data is coded as: 0, 1, 2 (default = `NULL`).
`resp`	A character with the name of the numeric variable in `pheno.data` with the response variable (default = `NULL`).
`weights`	A character with the name of the numeric variable in `pheno.data` with the weights of each prediction (i.e. mean estimate) (default = `NULL`).
`map.data`	A data frame with each marker's name, chromosome, and position. Variable names **must** be: "marker", "chrom", and "pos" (default = `NULL`).
`rename.markers`	If `TRUE` mathematical operators found on the markers' names will be replaced with the character "_" (default = `TRUE`).
`Q.method`	A character with the data source to be used for the calculation of the $\boldsymbol{Q}$ matrix by principal component analyses (PCA). Options are `"K"` (for kinship) and `"geno.data"` (for marker data) (default = `"K"`).
`K`	(Optional) A symmetric matrix representing the genomic relationship or kinship matrix $\boldsymbol{K}$. The matrices must be in full form with size $n \times n$ with n individuals (genotypes). If not provided, it will be calculated based on `geno.data` (default = `NULL`).
`return.K`	If `TRUE` the calculated $\boldsymbol{K}$ matrix will be included as part of the output (default = `FALSE`).
`message`	If `TRUE` diagnostic messages are printed on screen (default = `TRUE`).
`...`	Further arguments to be called from functions qc.filtering, snp.pruning, G.matrix, G.tuneup, kinship.diagnostics, G.inverse, kinship.pca, and snp.pca available from the package **ASRgenomics**.

Details

A summary of the workflow considered in `pre.gwas` is:

- 1: Rename markers (if required).
- 2: Remove individual records with `NA` on `resp` and `weights`.
- 3: Organize variables/columns in `pheno.data` (if necessary).
- 4: Pre-match `pheno.data` and `geno.data`.
- 5: Run qc.filtering on `geno.data` to obtain filtered marker matrix.
- 6: Round genotypic values on `geno.data` (if decimals are present) as required by G.matrix.
- 7: Obtain matrix $\boldsymbol{K}$ using function G.matrix, or use the one provided by user.
- 8: Run G.tuneup on $\boldsymbol{K}$ with blending, bending, and aligning (if requested by user).
- 9: Perform diagnostics on $\boldsymbol{K}$ matrix using kinship.diagnostics.
- 10: Repeat match of all datasets after $\boldsymbol{K}$ quality control.
- 11: Reorder all matrices based on `levels(pheno.data[[indiv]])`.
- 12: Obtain inverse of kinship matrix ($\boldsymbol{Kinv}$) using G.inverse.
- 13: If $\boldsymbol{Kinv}$ is ill-conditioned (changes are cumulative) then:
- 13.1: Run G.tuneup on $\boldsymbol{K}$ with blending of 0.05 using an identity matrix. If inverse is still ill-conditioned then go to 13.2; otherwise go to 14
- 13.2: Run G.tuneup on $\boldsymbol{K}$ again with blending of 0.10 using an identity matrix. If inverse is still ill-conditioned then stop; otherwise go to 14.
- 14: Obtain matrix $\boldsymbol{Q}$ based on `geno.data` or final $\boldsymbol{K}$ matrix.

Default values for arguments are used as suggested in **ASRgenomics**, but these can be modified via the ellipsis argument (`...`). Nevertheless, some defaults have been changed for better implementation of GWAS downstream procedures. The modified defaults are: `ncp = 20`, `maf = 0.05`, `marker.callrate = 0.2`, `ind.callrate = 0.2`, `clean.diagonal = TRUE`, `clean.duplicate = TRUE`, `diagonal.thr.large = 1.8`, and `diagonal.thr.small = 0.2`. It might be important to adjust these parameters along with GWAS runs to identify best procedures.

The $\boldsymbol{Q}$ matrix can be calculated based on $\boldsymbol{K}$ or `geno.data`. Nevertheless, if missing values are still present in `geno.data` after cleaning, `Q.method = "K"` will be always used, as this method allows for missing values.

For more information on the aforementioned functions, please refer to the **ASRgenomics** help.

Value

A list with several data frames and objects to be used in downstream GWAS model fits.

- `pheno.data`: A verified phenotypic data frame with all relevant columns (and rows) and responses to be used.
- `map.data`: A verified (or generated) data frame with marker's name, chromosome, and position.
- `geno.data`: A filtered and verified matrix with marker data.
- `K`: A verified and tuned-up kinship matrix (if requested).
- `Kinv`: A verified and tuned-up inverse of the kinship matrix in sparse format (ready for ASReml-R).
- `Q`: A population structure $\boldsymbol{Q}$ matrix of principal component scores.
- `plot.scree`: The scree plot for the components in the $\boldsymbol{Q}$ matrix.
- `eigenvalues`: The eigenvalues for the components in the $\boldsymbol{Q}$ matrix.

Examples

```
# Check additional arguments from ASRgenomics (parsed with ...).
pre.gwas()

# Prepare Apricot datasets (simple run).
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot)

# Prepare Apricot datasets (adding more filters/procedures).
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  weights = "Fructose", geno.data = geno.apricot, map.data = map.apricot,
  Q.method = "geno.data",
  # For ASRgenomics.
  maf = 0.05, marker.callrate = 0.40, pruning.thr = 0.95, by.chrom = TRUE,
  clean.diagonal = FALSE, impute = TRUE)
ls(gwas.data)
gwas.data$plot.scree
head(gwas.data$eigenvalues)
```

---

qq.plot *Draws one or more Q-Q plots for GWAS marker data results*

Description

Generates one or more quantile-quantile (Q-Q) plots for visualization.

Usage

```
qq.plot(
  gwas.table = NULL,
  point.colour = "#0072B2",
  point.size = 1,
  point.alpha = 1,
  trend.colour = "grey",
  coord.equal = TRUE,
  gwas.index = NULL,
  facet.main = NULL,
  facet.secondary = NULL,
  facet.dim = NULL,
  legend.position = "none",
  message = TRUE
)
```

Arguments

gwas.table	A data frame of one or more (vertically stacked) GWAS analyses with (as a minimum) the "p.value" column of marker results (default = NULL).
point.colour	A character with a colour name (e.g. "blue") or a column name from gwas.table to be used as the index for point colouring (default = "#0072B2").
point.size	A numeric value (greater than 0) indicating the point size (default = 1).

point.alpha	A numeric value indicating the points' transparency (default = 1).
trend.colour	A character with a colour name (e.g. "blue") to be used for colouring of the central line (expected value) (default = "black").
coord.equal	If TRUE the x and y coordinates will have the same limits and a ratio of 1 (default = TRUE).
gwas.index	A character indicating a column name in gwas.table identifying each GWAS analysis. This is only necessary when more than one GWAS analysis is present in gwas.table (default = NULL).
facet.main	A character indicating a column name in gwas.table to be used for the main faceting of panels (if more than one GWAS analysis is present) (default = NULL).
facet.secondary	A character indicating a column name in gwas.table to be used as secondary faceting of panels (if more than one GWAS analysis is present) (default = NULL).
facet.dim	A numeric vector indicating the number of rows and columns to be used in the faceting process, e.g. c(4, 1), for 4 rows and 1 column. This argument only works if facet.main is provided, and works best if facet.secondary is NULL (default = NULL).
legend.position	A character indicating the position for the legend in the displayed plot. Options are: "bottom", "top", "left", "right", "none", or the corresponding vector of coordinates (such as c(0.95, 0.05) for x and y, respectively) (default = "none").
message	If TRUE diagnostic messages are printed on screen (default = TRUE).

Value

A ggplot objects containing one or more Q-Q plots.

Examples

```
# Prepare Apricot dataset.
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot, map.data = map.apricot, maf = 0.05,
  marker.callrate = 0.40, impute = TRUE)

# Run GWAS.
gwasA <- gwas.asreml(
  pheno.data = gwas.data$pheno.data, resp = "Sucrose", gen = "Ind",
  fixedf = "Lots", residual = "Lots",
  Kinv = gwas.data$Kinv, Q = gwas.data$Q, npc = 3,
  geno.data = gwas.data$geno.data, map.data = gwas.data$map.data,
  pvalue.thr = 0.0005, workspace = "2Gb")

# Simple QQ plot call.
qq.plot(gwas.table = gwasA$gwas.all)

# Manipulate some features.
qq.plot(
  gwas.table = gwasA$gwas.all, point.colour = "green",
  point.alpha = 0.5, point.size = 2)
```

`select.marker` *Selects final set of GWAS markers using backward elimination*

Description

The backward elimination technique starts by adding all markers to the base model as covariates and obtaining their $p-value$. The marker with the highest $p-value$ is eliminated and the model is re-fit. The process is repeated until all retained markers have a $p-value$ smaller than the provided threshold.

Usage

```
select.marker(
  gwas.object = NULL,
  geno.data.sel = NULL,
  ref.vc = 1,
  pvalue.thr = 0.1,
  maxiter.update = 5,
  workspace = "1Gb",
  message = TRUE
)
```

Arguments

`gwas.object`	A (list) with all objects obtained after running the function gwas.asreml (default = `NULL`).
`geno.data.sel`	A matrix with (a subset of) marker data of form $n \times s$, with n individuals and s "significant" markers to be evaluated in the backward selection process. Individuals and marker names are assigned to `rownames` and `colnames`, respectively. Markers can have any coding (e.g., additive or dominant) as they will be fitted as covariates in the model (default = `NULL`).
`ref.vc`	A numeric (vector) containing the indices (or positions) of the variance components (from `gwas.object$mod$vparameters`) to be used as the reference for the variance explained by the markers (usually the genetic variance) (default = `1`).
`pvalue.thr`	A numeric value with the corresponding $p-value$ threshold to identify significant markers for the elimination process. Markers will be eliminated until all have a $p-value$ smaller than the value provided (default = `0.1`)
`maxiter.update`	A numeric value indicating the number of iterations to be used in update.asreml (default = `3L`).
`workspace`	Specifies the workspace to be used by the `asreml` function (default = `"1Gb"`).
`message`	If `TRUE` diagnostic messages are printed on screen (default = `TRUE`).

Details

If there is a group of markers with high collinearity (i.e., correlation >= 0.95) then the process stops and the list of markers that are highly correlated is reported. Here, one or more of these markers needs to be eliminated for the backward elimination to proceed.

Value

A list containing:

- `call`: An object of class `call` with the code used to fit `mod`.
- `gwas.sel`: A vector with the name of final marker set with `p-value < pvalue.thr`.
- `mod`: The GWAS model fitted with the retained markers. Contains the same set of columns as `gwas.sel` from gwas.asreml.
- `aov`: The ANOVA table of the updated model with the final marker set.
- `expl.var`: The percentage of the genetic variance explained by the final marker set.
- `collinear.markers` [returned if collinearity is an issue]: A data frame with the name and correlation value of highly collinear/correlated markers that might affect model fitting.

Examples

```
# Prepare Apricot dataset.
gwas.data <- pre.gwas(
  pheno.data = pheno.apricot, indiv = "Ind", resp = "Sucrose",
  geno.data = geno.apricot, map.data = map.apricot, maf = 0.05,
  marker.callrate = 0.40, impute = TRUE)

# Run GWAS.
gwasA <- gwas.asreml(
  pheno.data = gwas.data$pheno.data, resp = "Sucrose", gen = "Ind",
  fixedf = "Lots", residual = "Lots",
  Kinv = gwas.data$Kinv, Q = gwas.data$Q, npc = 3,
  geno.data = gwas.data$geno.data, map.data = gwas.data$map.data,
  pvalue.thr = 0.0005, bonferroni = FALSE, workspace = "2Gb")

# Get name of significant markers.
sign.markers <- gwasA$gwas.sel$marker
geno.data.sel <- gwas.data$geno.data[, sign.markers]

# Identify the index of genetic variance components.
gwasA$mod$vparameters

# Run of marker selection (this will return only the correlations).
set <- select.marker(
  gwas.object = gwasA, geno.data.sel = geno.data.sel,
  ref.vc = 1, pvalue.thr = 0.1)
set$call
set$aov
```

Index